

Documento de Despliegue

GrooveLogs

Aplicación Web Progresiva de Gestión y Valoración
Musical



Nombre del fichero:	DAW-PWR-PWA-DISCOGS-2025 5. Documento de despliegue.pdf
Fecha de esta versión:	17/02/2026 18:07:00

Historial de revisiones

Fecha	Descripción	Autor
17/02/26	Creación del documento de despliegue	Sara Monzón Quesada

Contenido

1. Introducción.....	4
2. Detalles de las mejoras	4
2.1 Mejoras técnicas	4
2.2 Mejoras funcionales.....	4
3. Servidor de aplicaciones	5
3.1 Identificación del servidor	5
3.2 Requisitos mínimos.....	5
3.3 Configuración recomendada	5
4. Servidor de base de datos	6
4.1 Identificación del servidor	6
4.2 Requisitos mínimos.....	6
4.3 Configuración de conexión	7
4.4 Scripts	7
5. Ficheros o binarios a desplegar	7
5.1 Ubicación	7
Backend	7
Frontend	8
6. Operativa	8
6.1 Operaciones de base de datos.....	8
6.2 Operaciones de despliegue	8
6.3 Condiciones y verificación de éxito	10
7. Control de versiones	10
8. Observaciones	11

1. Introducción

La presente release corresponde a la versión 1.0 de la aplicación GrooveLogs, una Aplicación Web Progresiva (PWA) desarrollada con arquitectura cliente-servidor.

Esta versión incluye:

- Sistema de registro y autenticación mediante JWT.
- Gestión de usuarios con control de roles (USER / ADMIN).
- Gestión de favoritos.
- Sistema de valoraciones (ratings).
- Persistencia en base de datos MySQL.
- Auditoría automática (created_at, updated_at, created_by).
- Soft delete en usuarios.
- Política de privacidad integrada.

Se trata del primer despliegue estable en entorno de producción.

2. Detalles de las mejoras

2.1 Mejoras técnicas

- Implementación de autenticación stateless mediante JWT.
- Expiración automática del token a 24 horas.
- Validación de usuario en cada petición.
- Sistema de auditoría con @CreatedDate y @LastModifiedDate.
- Implementación de soft delete.
- Control de transacciones en capa de servicios.
- Separación en arquitectura en capas (Controller → Service → Repository).
- Protección de endpoints mediante Spring Security.
- Frontend desacoplado consumiendo API REST.

2.2 Mejoras funcionales

- Registro con aceptación de política de privacidad.
- Sistema de favoritos evitando duplicados.
- Sistema de valoraciones con actualización automática si ya existe.
- Perfil editable por el usuario.

- Diferenciación de roles ADMIN / USER.

3. Servidor de aplicaciones

3.1 Identificación del servidor

- Nombre del servicio: groovelogs-backend
- Proveedor: Render
- Tipo de despliegue: Contenedor Docker
- Dominio público: <https://groovelogs.onrender.com>
- Puerto expuesto por la aplicación: 8080
- Región: Europa (gestionada por Render)

El backend se encuentra desplegado como un servicio web Dockerizado, accesible públicamente mediante HTTPS.

3.2 Requisitos mínimos

El servidor de aplicaciones se basa en un contenedor Docker que incluye todos los requisitos necesarios para la ejecución del backend. Entorno de ejecución:

- Docker 20.x o superior
- Sistema operativo Linux (gestionado por Render)
- Java 17 (OpenJDK – Eclipse Temurin)
- Spring Boot con Tomcat embebido
- Maven Wrapper incluido en el proyecto

Recursos mínimos recomendados:

- Memoria RAM: 512 MB
- Memoria recomendada: 1 GB
- CPU: 1 vCPU
- Puerto disponible: 8080

3.3 Configuración recomendada

El backend utiliza un Dockerfile multi-stage, optimizando el proceso de compilación y ejecución. Fase de compilación (Build)

- Imagen base: eclipse-temurin:17-jdk
- Se copian dependencias y código fuente.
- Se ejecuta Maven Wrapper en modo offline.
- Se genera el archivo .jar ejecutable.

Fase de ejecución (Runtime)

- Imagen base: eclipse-temurin:17-jre
- Se copia únicamente el .jar generado.
- Se expone el puerto 8080.

La aplicación se inicia mediante: `java -jar app.jar`

Este enfoque reduce el tamaño final de la imagen y mejora la seguridad al no incluir herramientas de compilación en producción. Las siguientes variables se definen directamente en el panel de Render:

Variable	Descripción
SPRING_DATASOURCE_URL	URL JDBC de PostgreSQL (Neon)
SPRING_DATASOURCE_USERNAME	Usuario de la base de datos
SPRING_DATASOURCE_PASSWORD	Contraseña de la base de datos
PORT	Puerto de la aplicación

4. Servidor de base de datos

4.1 Identificación del servidor

- Motor: PostgreSQL
- Proveedor: Neon
- Tipo: Base de datos serverless en la nube
- Nombre del proyecto: groovelogs
- Branch activo: production
- Región: AWS Europe West (Londres)
- Conexión segura: SSL obligatoria

La base de datos se gestiona de forma externa y no requiere instalación ni mantenimiento del motor en el servidor de aplicaciones.

4.2 Requisitos mínimos

- PostgreSQL versión 14 o superior
- Conexión TCP/IP segura
- Soporte SSL habilitado
- Acceso mediante credenciales configuradas como variables de entorno

4.3 Configuración de conexión

La conexión entre el backend y la base de datos se realiza mediante una URL JDBC proporcionada por Neon:

```
jdbc:postgresql://<host>.neon.tech:5432/groovelogs?sslmode=require
```

Configuración utilizada por Spring Boot:

El esquema de la base de datos se genera automáticamente a partir de las entidades JPA en el primer arranque de la aplicación.

```
spring.datasource.driver-class-name=org.postgresql.Driver  
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect  
spring.jpa.hibernate.ddl-auto=update
```

4.4 Scripts

No es necesaria la ejecución manual de scripts SQL, ya que:

- Hibernate gestiona la creación y actualización del esquema.
- Las entidades definen completamente la estructura de la base de datos.

5. Ficheros o binarios a desplegar

5.1 Ubicación

El despliegue se realizará a partir de un repositorio remoto en

<https://github.com/SaraMonzonDAW/groovelogs>

El proyecto se estructura como un monorepo, con la siguiente estructura básica:

```
groovelogs/  
├── backend/  
│   ├── Dockerfile  
│   ├── pom.xml  
│   ├── mvnw  
│   └── src/  
├── frontend/  
│   ├── package.json  
│   └── src/  
└── README.md
```

Backend

El backend no requiere subir manualmente archivos .jar, ya que Render construye automáticamente la imagen Docker a partir del Dockerfile.

No obstante, el artefacto generado es: backend/target/GrooveLogs-0.0.1-SNAPSHOT.jar

Este archivo es generado automáticamente durante el proceso de build dentro del contenedor Docker.

Frontend

El frontend se compila mediante: `npm run build`, generando los archivos estáticos en: `frontend/dist/`. Estos archivos pueden desplegarse como servicio estático en Render o mediante servidor web como Nginx, pero en nuestro caso lo estamos desplegando desde la propia página de github.

6. Operativa

6.1 Operaciones de base de datos

La base de datos está alojada en Neon (PostgreSQL serverless). Procedimiento:

1. Crear proyecto en Neon.
2. Crear base de datos groovelogs.
3. Obtener cadena de conexión segura.
4. Verificar que SSL está habilitado.
5. Configurar las siguientes variables en Render:

```
SPRING_DATASOURCE_URL=jdbc:postgresql://<host>.neon.tech:5432/groovelogs?ssl  
mode=require
```

```
SPRING_DATASOURCE_USERNAME=<usuario>
```

```
SPRING_DATASOURCE_PASSWORD=<contraseña>
```

No es necesario ejecutar scripts SQL manuales, ya que Hibernate genera el esquema automáticamente en el primer arranque.

6.2 Operaciones de despliegue

El backend se desplegará con Render más Docker:

1. Acceder a Render.
2. Crear nuevo Web Service.
3. Conectar con repositorio GitHub.
4. Seleccionar carpeta backend.
5. Render detecta automáticamente el Dockerfile.
6. Configurar variables de entorno.
7. Iniciar despliegue automático.

El backend quedará accesible en: <https://groovelogs.onrender.com/>

El frontend de GrooveLogs se despliega automáticamente mediante GitHub Actions, utilizando el servicio GitHub Pages como entorno de publicación.

Se ha configurado un workflow en el repositorio ubicado en: `.github/workflows/deploy-frontend.yml`. Este workflow permite automatizar el proceso de compilación y despliegue cada vez que se realiza un push a la rama principal (main).

```
on:  
  
  push:  
  
    branches: [ "main" ]
```

Tenemos en primer lugar una activación automática, donde el despliegue se ejecuta automáticamente cuando se realiza un push a la rama main. Esto garantiza que siempre que se publique una nueva versión estable, el frontend se actualice automáticamente.

```
runs-on: ubuntu-latest
```

En el entorno de ejecución el proceso se ejecuta en una máquina virtual Linux proporcionada por GitHub.

```
uses: actions/checkout@v4
```

Con la descarga del repositorio, se clona el código del repositorio en el entorno de ejecución.

```
- uses: actions/setup-node@v4  
  with:  
    node-version: 18
```

Instalación de Node.js, se instala Node.js versión 18, que es la versión mínima requerida por el proyecto.

```
working-directory: frontend  
run: |  
  npm install  
  npm run build
```

Instalación de dependencias y build, accede a la carpeta frontend y se instalan las dependencias definidas en package.json. Se genera la versión optimizada para producción en la carpeta: frontend/dist

```
uses: actions/upload-pages-  
artifact@v3  
  
uses: actions/deploy-pages@v4
```

Publicación en GitHub Pages: el directorio dist se sube como artefacto y posteriormente se despliega automáticamente en GitHub Pages.

6.3 Condiciones y verificación de éxito

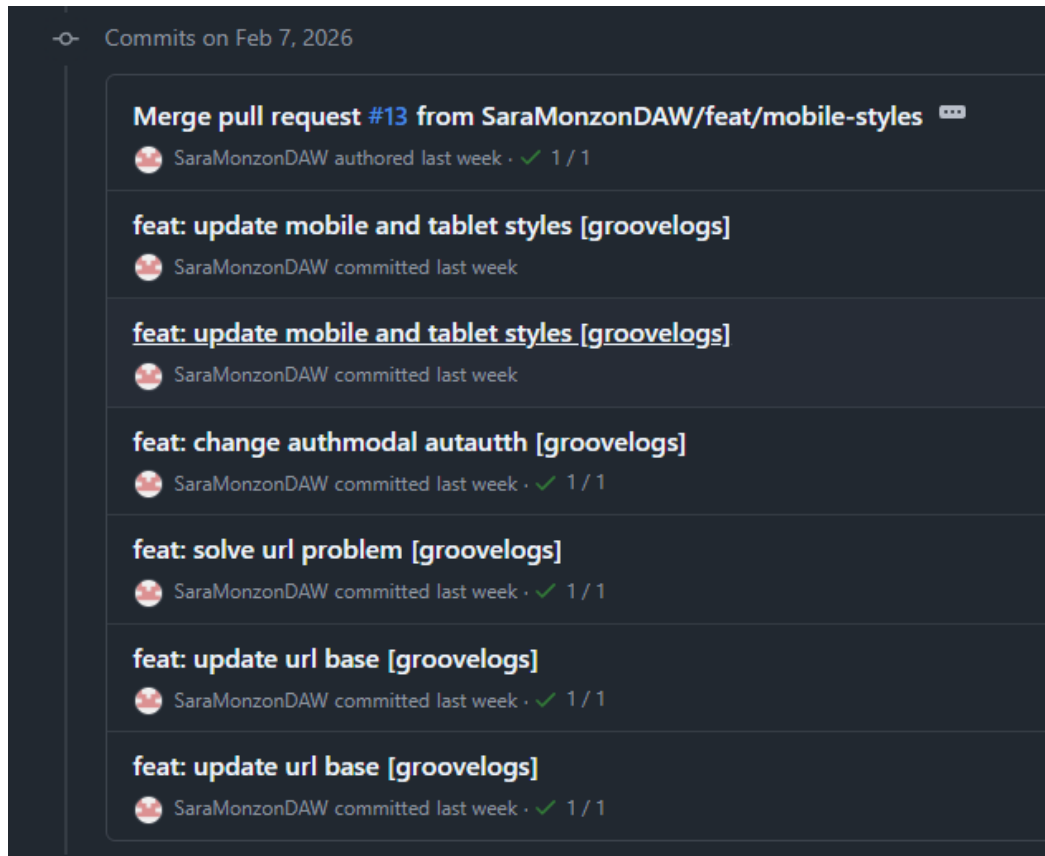
El despliegue se considera correcto cuando:

- El endpoint /auth/login responde con código 200.
- Swagger UI es accesible en: <https://groovelogs.onrender.com/swagger-ui/index.html>
- El frontend carga correctamente.
- Se puede registrar un nuevo usuario.
- Se crea registro en base de datos Neon.
- El token JWT se genera correctamente.
- Al eliminar un usuario, se produce soft delete.
- El sistema invalida sesión si el usuario es eliminado.

7. Control de versiones

Repositorio remoto oficial: <https://github.com/SaraMonzonDAW/groovelogs>

- Rama principal: main
- Control de versiones gestionado mediante Git.
- Historial de commits progresivo y siguiendo la estructura de conventional commits. Enlace al historial de commits: <https://github.com/SaraMonzonDAW/groovelogs/commits/main/> Imagen de ejemplo de algunos mensajes que se han ido añadiendo a la rama main del repositorio:



- Uso de mensajes descriptivos.
- Separación clara entre frontend y backend.

El despliegue corresponde a la versión estable 1.0.

8. Observaciones

- La aplicación utiliza autenticación JWT con expiración de 24 horas.
- Las credenciales sensibles no se almacenan en el repositorio.
- Todas las configuraciones sensibles se gestionan mediante variables de entorno.
- Se utiliza Docker multi-stage para optimizar el tamaño de la imagen.
- La base de datos se encuentra en infraestructura serverless (Neon).
- El proyecto sigue arquitectura en capas.
- Se implementa auditoría automática y soft delete.
- Se recomienda entorno Linux para despliegue.